

**LAPORAN PRATIKUM STRUKTUR DATA**  
**“QUEUE”**

**disusun Oleh:**  
**AFIF RAHMAN SALEH**  
**NIM 251153105**

**Dosen Pengampu: Dr. WAHYUDI, S.T, M.T**  
**Asisten Praktikum : M Galid Alvero**



**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**  
**TAHUN**  
**2026**

## **KATA PENGANTAR**

Puji syukur ke hadirat Allah SWT atas rahmat-Nya sehingga laporan praktikum Struktur Data ini dapat diselesaikan. Laporan ini disusun sebagai bentuk pertanggungjawaban kegiatan praktikum pekan pertama dengan topik implementasi Stack pada java.

Ucapan terima kasih penulis sampaikan kepada dosen pengampu, asisten praktikum, serta rekan-rekan mahasiswa atas bimbingan dan kerja sama yang diberikan. Penulis menyadari laporan ini masih jauh dari sempurna, sehingga kritik dan saran sangat diharapkan demi perbaikan di masa mendatang.

Semoga laporan ini bermanfaat bagi penulis maupun pembaca dalam memahami konsep struktur data dan penerapannya dalam pemrograman.

## DAFTAR ISI

|                                       |    |
|---------------------------------------|----|
| KATA PENGANTAR.....                   | i  |
| DAFTAR ISI .....                      | ii |
| BAB I PENDAHULUAN .....               | 1  |
| 1.1    Latar Belakang .....           | 1  |
| 1.2    Tujuan Praktikum.....          | 1  |
| 1.3    Manfaat Praktikum.....         | 1  |
| BAB II PEMBAHASAN .....               | 2  |
| 2.1 Landasan Teori .....              | 2  |
| 2.2 Kode Java yang Dipraktikkan ..... | 2  |
| 2.3 Penjelasan Langkah Kerja.....     | 5  |
| 2.4 Analisis Hasil .....              | 9  |
| BAB III KESIMPULAN.....               | 11 |
| 5.1    Kesimpulan Praktikum.....      | 11 |
| DAFTAR PUSTAKA .....                  | 12 |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Struktur data merupakan bagian penting dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara efisien. Salah satu struktur data yang sering digunakan adalah Queue, yaitu struktur data yang bekerja dengan prinsip FIFO (First In First Out).

Konsep ini banyak diaplikasikan dalam kehidupan nyata, seperti antrian di loket, proses penjadwalan tugas pada sistem operasi, maupun pengelolaan data dalam jaringan komputer.

### **1.2 Tujuan Praktikum**

1. Memahami konsep dasar Queue dan prinsip FIFO.
2. Mempelajari cara membuat, menambahkan (enqueue), dan menghapus (dequeue) elemen pada Queue di Java.
3. Mengimplementasikan penggunaan Queue dalam program sederhana.

### **1.3 Manfaat Praktikum**

1. Memberikan pemahaman lebih mendalam mengenai struktur data antrian di Java.
2. Membantu mahasiswa dalam menyelesaikan permasalahan pemrograman yang membutuhkan pengelolaan data secara berurutan.
3. Menjadi bekal dalam pengembangan aplikasi yang lebih kompleks, seperti sistem penjadwalan, simulasi antrian, dan manajemen proses.

## BAB II

### PEMBAHASAN

#### 2.1 Landasan Teori

Queue adalah struktur data linear yang menerapkan prinsip First In First Out (FIFO). Artinya, elemen yang pertama kali masuk ke dalam antrian akan menjadi elemen pertama yang keluar. Konsep ini mirip dengan antrian pada kehidupan sehari-hari, seperti orang yang mengantri di loket atau kendaraan yang mengantri di jalan tol.

Operasi dasar pada Queue:

- Enqueue → Menambahkan elemen ke dalam antrian.
- Dequeue → Menghapus elemen dari antrian.
- Peek/Front → Melihat elemen terdepan tanpa menghapusnya.
- isEmpty → Mengecek apakah antrian kosong.
- isFull → Mengecek apakah antrian penuh (pada implementasi berbasis array).

#### 2.2 Kode Java yang Dipraktikkan

##### 1. Kelas QueueArray\_2511531005

```
package pekan4_2511531005;

public class QueueArray_2511531005 {

    int front, rear, size;
    int capacity;
    int array[];

    public QueueArray_2511531005(int capacity) {
        this.capacity=capacity;
        front = this.size = 0;
        rear = capacity - 1 ;
        array = new int[this.capacity];
    }

    boolean isFull_1005(QueueArray_2511531005 queue) {
        return (queue.size == queue.capacity);
    }
}
```

```

boolean isEmpty_1005(QueueArray_2511531005 queue) {
    return (queue.size==0);
}

void enqueue_1005(int item) {
    if (isFull_1005 (this))
        return;
    this.rear = (this.rear +1) % this.capacity;
    this.array[this.rear] = item;
    this.size = this.size + 1;
    System.out.println(item + " endqueue to queue");
}

int dequeue_1005() {
    if(isEmpty_1005(this))
        return Integer.MIN_VALUE;
    int item = this.array[this.front];
    this.front = (this.front + 1 ) % this.capacity;
    this.size = this.size -1;
    return item;
}

int front_1005() {
    if(isEmpty_1005(this))
        return Integer.MIN_VALUE;
    return this.array[this.front];
}

int rear_1005() {
    if (isEmpty_1005(this))
        return Integer.MIN_VALUE;
    return this.array[this.rear];
}

void display_1005() {
    int i;
    if(front == rear) {
        System.out.printf("\nAntrian kosong\n");
        return;
    }

    for (i = front; i < rear ; i++) {
        System.out.printf(" %d <= ", array[i]);
    }
    return;
}
}

```

## 2. Kelas QueueArrayDriver\_2511531005

```

package pekan4_2511531005;

public class QueueArrayDriver_2511531005 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        QueueArray_2511531005 queue = new QueueArray_2511531005(1000);
        queue.enqueue_1005(10);
        queue.enqueue_1005(20);
        queue.enqueue_1005(30);
        queue.enqueue_1005(40);
        System.out.println("item di depan " + queue.front_1005());
        System.out.println("item paling belakang " + queue.rear_1005());
        System.out.println("tampilan queue");
        queue.display_1005();
        System.out.println();
    }
}

```

```

        System.out.println(queue.dequeue_1005() + " dihapus dari queue");
        System.out.println("item di depan : " + queue.front_1005());
        System.out.println("item di belakang : " + queue.rear_1005());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_1005();
    }
}

```

### 3. Kelas ReverseData\_2511531005

```

package pekan4_2511531005;
import java.util.*;
public class ReverseData_2511531005 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Queue<Integer> q = new LinkedList<Integer>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println("sebelum reverse " + q);
        Stack<Integer> s = new Stack<Integer>();
        while (!q.isEmpty()) {
            s.push(q.remove());
        }

        while (!s.isEmpty()) {
            q.add(s.pop());
        }
        System.out.println("sesudah reverse = " + q);
    }
}

```

### 4. Kelas IterasiQueue\_2511531005

```

package pekan4_2511531005;
import java.util.*;
public class IterasiQueue_2511531005 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Queue<String> q = new LinkedList<>();

        q.add("praktikum");
        q.add("struktur");
        q.add("data");
        q.add("dan");
        q.add("algoritma");
        Iterator<String> iterator = q.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next()+ " ");
        }
    }
}

```

### 5. Kelas QueueLinkedList\_2511531005

```

package pekan4_2511531005;

import java.util.*;

public class QueueLinkedList_2511531005 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Queue<Integer> q = new LinkedList<>();

        for (int i = 0; i<6; i++) {
            q.add(i);
        }

        System.out.println("elemen antrian " + q);

        int hapus = q.remove();
        System.out.println("hapus elemen = " + hapus);
        System.out.println(q);

        int depan = q.peek();
        System.out.println("kepala antrian = " + depan);

        int banyak = q.size();
        System.out.println("size antrian = " + banyak);

    }
}

```

## 2.3 Penjelasan Langkah Kerja

### 1. Kelas QueueArray\_2511531005

- Variable
  - Int front, rear, size : untuk menyimpan indeks depan, belakang dan jumlah elemen dalam queue
  - Int capacity : kapasitas maksimum quque
  - Int array[] : array untuk menyimpan quque

- Konstruktor

```

• public QueueArray_2511531005(int capacity) {
•     this.capacity=capacity;
•     front = this.size = 0;
•     rear = capacity - 1 ;
•     array = new int[this.capacity];
• }

```

- Front : dimulai dari 0
- Rear : diinisialisaikan sebagai (capacity – 1) agar saat di enqueue posisi rear akan 0
- Array : buat sesuai kapasitas

- Method utama



- isFull\_1005 (Mengecek apakah queue sudah penuh)

```
• boolean isFull_1005(QueueArray_2511531005 queue) {
•     return (queue.size == queue.capacity);
• }
```

- isEmpty\_1005 (mengecek apakah queue kosong)

```
• boolean isEmpty_1005(QueueArray_2511531005 queue) {
•     return (queue.size==0);
• }
```

- enqueue\_1005 (int item)

```
• void enqueue_1005(int item) {
•     if (isFull_1005 (this))
•         return;
•     this.rear = (this.rear +1) % this.capacity;
•     this.array[this.rear] = item;
•     this.size = this.size + 1;
•     System.out.println(item + " enqueue to queue");
• }
```

- menambah elemen ke belakang queue
- menghitung rear dengan  $(rear + 1) \% capacity$  agar circular
- size +1

- Dequeue\_1005

```
• int dequeue_1005() {
•     if(isEmpty_1005(this))
•         return Integer.MIN_VALUE;
•     int item = this.array[this.front];
•     this.front = (this.front + 1 ) % this.capacity;
•     this.size = this.size -1;
•     return item;
• }
```

- Menghapus elemen dari depan queue
- Front bergeser ke indeks berikutnya
- Size -1
- Jika kosong return Integer.MIN\_VALUE

- Front\_1005 dan rear\_1005

- Mengembalikan elemen paling depan (front) dan paling belakang(rear)

- Display\_1005

```
• void display_1005() {
•     int i;
•     if(front == rear) {
•         System.out.printf("\nAntrian kosong\n");
•         return;
•     }
• }
```

```

•         for (i = front; i < rear ; i++) {
•             System.out.printf(" %d <= ", array[i]);
•         }
•         return;
•     }

```

- Menampilkan isi queue dari front ke rear

## 2. Kelas QueueArrayDriver\_2511531005

- Main program untuk menguji queue

```

• QueueArray_2511531005 queue = new QueueArray_2511531005(1000);
• queue.enqueue_1005(10);
• queue.enqueue_1005(20);
• queue.enqueue_1005(30);
• queue.enqueue_1005(40);

```

- Membuat queue dengan kapasitas 1000
- Menambahkan elemen 10,20,30,40 ke dalam queue

```

• System.out.println("item di depan " + queue.front_1005());
• System.out.println("item paling belakang " + queue.rear_1005());
• System.out.println("tampilan queue");
• queue.display_1005();

```

- Menampilkan elemen depan dan belakang

```

• System.out.println(queue.dequeue_1005() + " dihapus dari queue");
• System.out.println("item di depan : " + queue.front_1005());
• System.out.println("item di belakang : " + queue.rear_1005());
• System.out.println("tampilan queue setelah satu data dihapus");
• queue.display_1005();

```

- menghapus elemen depan dari queue
- menampilkan queue setelah penghapusan

## 3. ReverseData\_2511531005

- Membuat queue

```

• Queue<Integer> q = new LinkedList<Integer>();
• q.add(1);
• q.add(2);
• q.add(3);

```

- Membuat queue dengan linkedlist
- Menambahkan elemen 1,2,3 ke dalam queue

- Membuat stack

```

Stack<Integer> s = new Stack<Integer>();

```

- Membuat stack kosong untuk membantu pembalikan data

- Memindahkan data dari queue ke stack

```

• while (!q.isEmpty()) {
•     s.push(q.remove());

```

- }
  - Selama queue tidak kosong
    - q.remove() : mengambil elemen depan queue
    - s.push() : menyimpan elemen ke dalam stack
- memindahkan elemen dari stack ke queue

```
• while (!s.isEmpty()) {
•     q.add(s.pop());
• }
```

- Selama stack tidak kosong
  - s.pop() : mengambil elemen depan stack
  - q.add () : menambah elemen ke dalam queue

#### 4. IterasiQueue\_2511531005

- Membuat queue

```
• Queue<String> q = new LinkedList<>();
•
```

- Membuat queue dengan linkedlist bertipedata string

- Menambahkan elemen queue

```
• q.add("praktikum");
• q.add("struktur");
• q.add("data");
• q.add("dan");
• q.add("algoritma");
```

- Membuat iterator

```
• Iterator<String> iterator = q.iterator();
•
```

- Membauat iterator untuk menelusuri queue
- Iterator akan mulai dari elemen pertama

- Iterasi dengan while loop

```
• while (iterator.hasNext()) {
•     System.out.print(iterator.next()+ " ");
• }
```

- Iterator.hasNext() : mengecek apakah masih ada elemen berikutnya
- Iterator.Next() : mengambil elemen saat ini dan bergerak ke elamen selanjutnyalanjutnya

#### 5. QueueLinkedList\_2511531005

- Membuat queue

```
• Queue<Integer> q = new LinkedList<>();
```

- Membuat queue dengan linkedlist bertipe data integer

- Menambahkan elemen ke dalam queue

```
• for (int i = 0; i<6; i++) {
•     q.add(i);
• }
```

- Loop menambahkan angka 0-5 ke dalam queue

- Menghapus elemen dari queue

```
• int hapus = q.remove();
•     System.out.println("hapus elemen = " + hapus);
•     System.out.println(q);
```

- Remove() : menghapus elemen depan queue

- Melihat elemen paling depan

```
• int depan = q.peek();
•     System.out.println("kepala antrian = " + depan);
```

- Peek() : mengembalikan elemen depan tanpa menghapusnya

- Menampilkan ukuran queue

```
• int banyak = q.size();
•     System.out.println("size antrian = " + banyak);
```

- Size() : mengembalikan jumlah elemen dalam queue

## 2.4 Analisis Hasil

Berdasarkan program pada semua kelas yang dipraktikkan pada pekan ini, diperoleh beberapa hasil sebagai berikut:

1. QueueArray\_2511531005 dan QueueArrayDriver\_2511531005

Output :

```
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
item di depan 10
item paling belakang 40
tampilan queue
10 <== 20 <== 30 <==
10 dihapus dari queue
item di depan : 20
item di belakang : 40
tampilan queue setelah satu data dihapus
20 <== 30 <==
```

Berdasarkan output di atas dapat disimpulkan proses enqueue dan dequeue berjalan dengan lancar serta front dan rear dapat menampilkan sesuai kondisi queue.

## 2. ReverseData\_2511531005

Output :

```
sebelum reverse [1, 2, 3]
sesudah reverse = [3, 2, 1]
```

Berdasarkan output di atas, konsep yang digunakan dalam program ini berupa queue(FIFO) dan stack(LIFO) yang mana dengan memindahkan data dari queue ke stack urutan data otomatis terbalik lalu dipindahkan lagi ke queue untuk mendapatkan hasil reversenya.

## 3. IterasiQueue\_2511531005

Output :

```
praktikum struktur data dan algoritma
```

Berdasarkan output di atas, program tersebut membuat sebuah queue berisi string dengan menggunakan iterator untuk menelusuri isi queue serta mencetak semua elemen queue secara berutan.

## 4. QueueLinkedList\_2511531005

Output :

```
elemen antrian [0, 1, 2, 3, 4, 5]
hapus elemen = 0
[1, 2, 3, 4, 5]
kepala antrian = 1
size antrian = 5
```

## **BAB III**

### **KESIMPULAN**

#### **1.1 Kesimpulan Praktikum**

Praktikum Queue pada Java membuktikan bahwa struktur data ini bekerja dengan prinsip *First In First Out (FIFO)* dan melalui implementasi sederhana mahasiswa dapat memahami operasi dasar seperti *enqueue*, *dequeue*, serta penerapannya dalam berbagai kasus nyata. Praktikum ini sekaligus melatih keterampilan pemrograman dan pemahaman konsep antrian dalam sistem komputer.

## DAFTAR PUSTAKA

- [1] Oracle. (2025). *Java Platform, Standard Edition Documentation*. Oracle Corporation.
- [2] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

## TUGAS PRAKTIKUM PEKAN 4

### A. Kode Program

```
• package pekan4_2511531005;
• import java.util.*;
• public class AntrianLoket_2511531005 {
•
•     private int front, rear, max;
•     private String [] queue;
•
•     public AntrianLoket_2511531005(int max) {
•         this.max = max;
•         queue = new String[max];
•         front = -1;
•         rear = -1;
•     }
•
•     boolean isEmpty() {
•         return front == -1;
•     }
•
•     boolean isFull() {
•         return rear == max -1;
•     }
•
•     void enqueue(String nama) {
•         if (isFull()) {
•             System.out.println("antrian penuh!");
•         }else {
•             if (front == -1) front = 0;
•             rear++;
•             queue[rear] = nama;
•             System.out.println("data berhasil ditambahkan
• ke antrian");
•         }
•     }
•
•     void dequeue() {
•         if (isEmpty()) {
•             System.out.println("antrian kosong!");
•         }else {
•             String served = queue[front];
•             if (front == rear) {
•                 front = -1;
•                 rear = -1;
•             }else {
•                 front++;
•             }
•             System.out.println(served + " telah
• dilayani");
•         }
•     }
• }
```



```

•
•
• void display() {
•     if(isEmpty()) {
•         System.out.println("antrian kosong!");
•     }else {
•         System.out.println("isi antrian : ");
•         int nomor =1;
•         for (int i = front ; i <= rear ; i++) {
•             System.out.println(nomor + ". " +
queue[i]);
•
•             nomor++;
•         }
•     }
• }
•
• void reverse() {
•     if(isEmpty()) {
•         System.out.println("antrian kosong!");
•         return;
•     }
•     int i = front;
•     int j = rear;
•     while(i<j) {
•         String temp = queue[i];
•         queue[i] = queue[j];
•         queue[j] = temp;
•         i++;
•         j--;
•     }
•     System.out.println("isi antrian :");
•     int nomor = 1;
•     for (int k = front ; k <= rear; k++){
•         System.out.println(nomor + ". " + queue[k]);
•         nomor++;
•     }
• }
•
• public static void main (String[] args) {
•
•     Scanner input = new Scanner (System.in);
•     AntrianLoket_2511531005 antrian = new
AntrianLoket_2511531005(10);
•
•     int choice;
•
•     do {
•         System.out.println("\n=== PROGRAM ANTRIAN
LOKET ===");
•
•         System.out.println("1. tambah antrian");
•         System.out.println("2. hapus antrian");
•         System.out.println("3. tampilkan antrian");
•         System.out.println("4. reverse");
•         System.out.println("5. keluar");

```

```

•         System.out.println("pilih menu : ");
•         choice = input.nextInt();
•         input.nextLine();
•
•
•         switch (choice) {
•         case 1:
•             System.out.println("masukkan nama
pelanggan : ");
•             String nama = input.nextLine();
•             antrian.enqueue(nama);
•             break;
•         case 2:
•             antrian.dequeue();
•             break;
•         case 3:
•             antrian.display();
•             break;
•         case 4:
•             antrian.reverse();
•             break;
•         default:
•             System.out.println("pilihan tidak
valid!");
•             break;
•         }
•     }
•     while (choice !=5);
•     input.close();
•
• }
•
• }

```

## B. Penjelasan Kode

Program ini bertujuan untuk membuat sistem antrian loket menggunakan array dengan kapasitas tertentu. Program ini memiliki operasi dasar antrian seperti enqueue (menambahkan pelanggan ke antrian), dequeue (menghapus pelanggan dari antrian), display (menampilkan isi antrian), dan reverse (membalik urutan antrian). Variabel front dan rear digunakan untuk menandai posisi awal dan akhir antrian, sedangkan metode isEmpty() dan isFull() mengecek kondisi antrian. Pada bagian main, program menyediakan menu interaktif berbasis input pengguna untuk memilih operasi antrian, sehingga dapat digunakan untuk menambah, melayani, menampilkan, atau membalik urutan pelanggan dalam antrian loket..

## C. Output Program

### 1. Tambah antrian

```
=== PROGRAM ANTRIAN LOKET ===
1. tambah antrian
2. hapus antrian
3. tampilkan antrian
4. reverse
5. keluar
pilih menu :
1
masukkan nama pelanggan :
arif
data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. tambah antrian
2. hapus antrian
3. tampilkan antrian
4. reverse
5. keluar
pilih menu :
1
masukkan nama pelanggan :
bayu
data berhasil ditambahkan ke antrian
```

### 2. Tampilkan antrian

```
=== PROGRAM ANTRIAN LOKET ===
1. tambah antrian
2. hapus antrian
3. tampilkan antrian
4. reverse
5. keluar
pilih menu :
3
isi antrian :
1. arif
2. bayu
```

### 3. Reverse antrian

```
=== PROGRAM ANTRIAN LOKET ===
1. tambah antrian
2. hapus antrian
3. tampilkan antrian
4. reverse
5. keluar
pilih menu :
4
isi antrian :
1. bayu
2. arif
```

4. Hapus antrian dapan

```
=== PROGRAM ANTRIAN LOKET ===  
1. tambah antrian  
2. hapus antrian  
3. tampilkan antrian  
4. reverse  
5. keluar  
pilih menu :  
2  
bayu telah dilayani
```

5. Lihat antrian setelah dihapus

```
=== PROGRAM ANTRIAN LOKET ===  
1. tambah antrian  
2. hapus antrian  
3. tampilkan antrian  
4. reverse  
5. keluar  
pilih menu :  
3  
isi antrian :  
1. arif
```

6. Program selesai

```
=== PROGRAM ANTRIAN LOKET ===  
1. tambah antrian  
2. hapus antrian  
3. tampilkan antrian  
4. reverse  
5. keluar  
pilih menu :  
5  
program selesai
```